

INTRODUCTION AUX SGBDR

COURS 1

Table des matières

Cours 1 : Systèmes de Gestion de Bases de Données Relationnelles (SGBDR).....	2
Objectifs du cours.....	2
1) Introduction aux bases de données.....	2
1.1 Définition claire : qu'est-ce qu'une base de données ?	2
1.2 Rôle et importance dans les systèmes informatiques modernes.....	2
2) Différence fondamentale entre donnée et information	3
2.1 Définitions précises	3
2.2 Exemples concrets.....	3
2.3 Comment les données deviennent de l'information ?	3
3) Historique des SGBDR	4
3.1 Origines et contexte	4
3.2 Le tournant : Edgar F. Codd et le modèle relationnel	4
3.3 Dates importantes (chronologie simple).....	4
4) Utilité d'une base de données	5
4.1 Pourquoi utiliser une base de données ?	5
4.2 Avantages principaux (avec explications simples)	5
4.3 Cas d'utilisation concrets (secteurs).....	6
5) Quand implémenter un SGBD dans un projet informatique ?	6
5.1 Le bon réflexe : "Est-ce que j'en ai vraiment besoin ?	6
5.2 Critères de décision (clairs et pratiques).....	6
5.3 Signaux d'alerte : "les fichiers plats ne suffisent plus".....	7
5.4 Alternatives légères : SQLite vs solutions d'entreprise	8

Cours 1 : Systèmes de Gestion de Bases de Données Relationnelles (SGBDR)

Objectifs du cours

À la fin, tu dois pouvoir :

- expliquer clairement ce qu'est une base de données et à quoi elle sert ;
- distinguer **donnée** et **information** avec des exemples concrets ;
- raconter l'**historique** des SGBDR (les grandes idées et les dates clés) ;
- décider **quand** un projet a besoin d'un SGBD relationnel (et quand un stockage plus simple suffit) ;
- comprendre les bases du **modèle relationnel** (tables, lignes, colonnes, relations).

1) Introduction aux bases de données

1.1 Définition claire : qu'est-ce qu'une base de données ?

Une **base de données** est un **ensemble organisé de données** stockées de manière structurée pour pouvoir :

- **ajouter** des données,
- **rechercher** rapidement,
- **modifier**,
- **supprimer**,
- **sécuriser**,
- **partager** entre plusieurs utilisateurs ou applications.

Analogie simple :

- Un **cahier** où tu notes des informations = stockage.
- Une **bibliothèque bien classée** (titres, auteurs, catégories, index) = base de données.

Dans une bibliothèque, tu retrouves vite un livre car tout est **organisé**.

1.2 Rôle et importance dans les systèmes informatiques modernes

Dans la vraie vie, presque toutes les applications reposent sur des données :

- un site e-commerce : produits, stocks, clients, commandes
- une école : étudiants, notes, absences, emplois du temps
- une banque : comptes, transactions
- une entreprise : RH, paie, projets, documents

Sans base de données, l'application devient vite :

- lente,
- difficile à maintenir,
- risquée (perte de données),
- limitée pour le travail en équipe.

2) Différence fondamentale entre donnée et information

2.1 Définitions précises

- **Donnée (data)** : un **fait brut**, isolé, sans contexte.
- **Information** : une **donnée interprétée** (traitée + contextualisée) qui devient utile pour décider.

2.2 Exemples concrets

Exemple 1 (école)

- Données : 12, 15, 8
- Informations : “La moyenne de l'étudiant A en base de données est **11,67/20**.”

Exemple 2 (commerce)

- Donnée : 250000
- Information : “Le chiffre d'affaires de janvier est de **250 000 FCFA**, en hausse de **12%** par rapport à décembre.”

Exemple 3 (santé)

- Données : 38.9, toux, fatigue
- Information : “Le patient présente une **fièvre** ; suspicion d'infection.”

2.3 Comment les données deviennent de l'information ?

Les données deviennent une information quand on applique un **traitement** :

- calcul (moyenne, somme, pourcentage)
- tri / filtrage (“les commandes de cette semaine”)

- regroupement (“ventes par ville”)
- comparaison (“par rapport au mois dernier”)
- mise en contexte (date, personne, lieu)

Formule simple à retenir :

Information = Données + Contexte + Traitement + Interprétation

3) Historique des SGBDR

3.1 Origines et contexte

Au début de l’informatique (années 1950–1960), les données étaient souvent stockées dans des **fichiers** (bandes, puis fichiers plats).

Problèmes majeurs :

- duplication des données
- incohérences (“Mansour” écrit différemment selon les fichiers)
- recherche lente
- difficulté à faire évoluer le système

Il fallait un modèle plus propre, plus “scientifique”, plus robuste.

3.2 Le tournant : Edgar F. Codd et le modèle relationnel

En 1970, Edgar F. Codd (chez IBM) propose le **modèle relationnel** :

représenter les données sous forme de relations (aujourd’hui : des tables).

Cette idée devient fondatrice car elle apporte :

- une structure claire
- des règles de cohérence
- un langage de requête standardisable (SQL)

3.3 Dates importantes (chronologie simple)

- **Années 1960** : bases de données hiérarchiques et réseau (avant le relationnel).
- **1970** : Codd formalise le **modèle relationnel**.
- **Années 1970** : prototypes (System R chez IBM, Ingres à l’université).
- **Fin 1970s – 1980s** : apparition et industrialisation des SGBDR.

- **1977 : Lawrence Ellison** cofonde l'entreprise qui lancera **Oracle**, l'un des SGBDR majeurs.
- **1986/1987** : normalisation de **SQL** (standard ANSI/ISO), ce qui accélère l'adoption.

L'idée clé : le relationnel + SQL a gagné parce que c'était plus simple, plus robuste et mieux standardisé.

4) Utilité d'une base de données

4.1 Pourquoi utiliser une base de données ?

Parce qu'une base de données permet de gérer correctement :

- la **persistance** (les données restent après fermeture de l'application)
- la **recherche rapide** (index, optimisation)
- la **cohérence** (contraintes)
- le **partage multi-utilisateurs** (accès concurrent)
- la **sécurité** (droits, rôles, traçabilité)
- la **sauvegarde** et la **restauration**

4.2 Avantages principaux (avec explications simples)

a) **Centralisation**

Tout est au même endroit : une "source de vérité".

- Moins de doublons
- Moins de versions contradictoires

b) **Intégrité (qualité des données)**

Le SGBDR impose des règles :

- un étudiant doit avoir un identifiant unique
- une note doit être entre 0 et 20
- une commande doit appartenir à un client existant

c) **Sécurité**

Tu peux dire :

- "tel utilisateur peut lire"
- "tel utilisateur peut modifier"

- “tel autre n’a accès qu’à ses propres données”

d) **Accès concurrent (multi-utilisateur)**

Plusieurs personnes peuvent travailler en même temps **sans casser** les données.

Le SGBDR gère les conflits via des mécanismes de **transactions**.

e) **Transactions (fiabilité)**

Une action complexe est **tout ou rien** :

- Exemple bancaire : débiter A **et** créditer B.

Si une partie échoue, on annule tout (pas de demi-erreur).

4.3 Cas d’utilisation concrets (secteurs)

- **Éducation** : gestion étudiants, notes, absences, emplois du temps.
- **Santé** : dossiers patients, rendez-vous, pharmacie, stocks.
- **Commerce** : produits, panier, commandes, paiements.
- **Administration** : état civil, dossiers, archives, suivi.
- **Transport/Logistique** : expéditions, suivi, planning, facturation.
- **Réseaux sociaux / médias** : utilisateurs, publications, commentaires.

5) Quand implémenter un SGBD dans un projet informatique ?

5.1 Le bon réflexe : “Est-ce que j’en ai vraiment besoin ?

On n’a pas besoin d’un gros SGBDR pour tout.

Mais dès qu’un système grandit, le stockage “simple” devient fragile.

5.2 Critères de décision (clairs et pratiques)

1) Volume de données

- quelques dizaines/centaines : un simple fichier peut suffire
- milliers/millions : base de données recommandée
- croissance prévue : pense à l’avenir

2) Complexité des relations

Si tu as plusieurs “entités” qui se connectent entre elles :

- Étudiants ↔ Classes ↔ Modules ↔ Notes
- Clients ↔ Commandes ↔ Produits ↔ Paiements

Alors un SGBDR est naturel.

3) Besoin de requêtes avancées

Si tu dois souvent répondre à :

- “les 10 meilleurs clients de ce mois”
- “la moyenne par classe”
- “les produits les plus vendus”
- “les rendez-vous de demain par médecin”

=> base de données.

4) Besoin de partage et accès concurrent

Plusieurs utilisateurs en même temps (admin + agents + clients) :

- un fichier devient vite un point de conflit
- un SGBDR gère ça proprement.

5) Exigences de sécurité et traçabilité

Si tu dois gérer :

- rôles (admin, agent, utilisateur)
- logs (qui a modifié quoi ?)
- données sensibles

=> SGBDR fortement conseillé.

5.3 Signaux d’alerte : “les fichiers plats ne suffisent plus”

Tu sais qu’il te faut un SGBD quand :

- tu as plusieurs fichiers “clients_v1.csv”, “clients_final.csv”, “clients_final2.csv”
- tu copies/colles beaucoup
- tu as des incohérences (“même client, 2 numéros différents”)
- la recherche devient lente

- tu n'arrives pas à empêcher les erreurs (dates invalides, doublons)
- plusieurs personnes doivent modifier les mêmes données

5.4 Alternatives légères : SQLite vs solutions d'entreprise

SQLite (léger)

- Base intégrée dans un fichier (souvent .db)
- Très pratique pour :
 - appli mobile locale
 - petite appli mono-utilisateur
 - prototype / démo
- Limites :
 - concurrence multi-utilisateur plus délicate à grande échelle
 - administration avancée moindre

SGBDR “serveur” (PostgreSQL, MySQL/MariaDB, Oracle, SQL Server...)

- Pensés pour :
 - multi-utilisateurs
 - grosses données
 - sécurité avancée
 - performance à grande échelle
- Choix typique pour une application web d'entreprise.